MORF: A Distributed Multimodal Information Filtering System

Yi-Leh Wu[†], Edward Y. Chang[§†], Kwang-Ting Cheng[§†],
Cheng-Wei Chang[†], Chen-Cha Hsu[†], Wei-Cheng Lai[†], and Ching-Tung Wu[†]

[†]Morpho Software Inc., 3944 State Steet, Suite 340, Santa Barbara, CA 93105
[§]Electrical & Computer Engineering, University of California, Santa Barbara, CA 93106

Email: echang@ece.ucsb.edu

Abstract. The proliferation of objectionable information on the Internet has reached a level of serious concern. To empower end-users with the choice of blocking undesirable and offensive Web-sites, we propose a multimodal personalized information filter, named *MORF*. The design of *MORF* aims to meet three major performance goals: efficiency, accuracy, and personalization. To achieve these design goals, we have devised a multimodality classification algorithm and a personalization algorithm. Empirical study and initial statistics collected from the *MORF* filters deployed at sites in the U.S. and Asia show that *MORF* is both efficient and effective, compared to the traditional URL- and text-based filtering approaches.

1 Introduction

Internet is a mixed blessing for home, schools, and workplace. While Internet access has opened up enormous new possibilities, it has also created new hazards in loss of productivity, bandwidth clogging, and corporate or institutional liability. To screen out objectionable information[1], many commercial filtering products have been developed. Most products, however, suffer from low filtering accuracy and lack of personalization capability. In this paper we present *MORF* (Multimodal Objectionable infoRmation Filter), a multimodal (including URL, texts, and images) personalized information filter. *MORF* employs our proposed multimodal classification and personalization algorithms to achieve efficient, effective, and personalized filtering.

The most widely used filters employs the URL-based technique. These filters maintain a "block" list of Web-site addresses. When a user requests a URL, the system checks to see whether the requested URL is on the block list. If yes, the filter refuses the request; otherwise, the request is granted. The drawback of the URL-based techniques is that the block list can quickly become outdated (according to a report from National Research Counsel, there are 300 to 400 new pornography sites created daily), and hence the filtering accuracy can be low.

The other popular filtering method is based on analyzing the text. A text-based technique analyzes the words in the user's requested content and maintains a list of words that commonly appear at objectionable Web-sites. A text-based filter matches words in the input text with the keywords on the bad-word list. If offensive keywords appear frequently above a tolerable threshold, the Web-site containing the words is regarded as objectionable and blocked. The text-based approach has at least three shortcomings. First, the matching process is language dependent. Second, the lexical meanings of keywords can be ambiguous. For example, some words can be both objectionable and benign depending on their context (e.g. wren tits, wood screws, breast cancer). Third, many objectionable sites (e.g., pornography sites) embed words in images, and therefore, no text is available for text-based analysis.

To overcome the low-accuracy shortcoming of the traditional URL- and text-based approaches while maintaining high filtering efficiency (the filtering activities should not significantly affect network access time and throughput), *MORF* employs a multimodal filtering algorithm. The benefits of this approach can be summarized as follows:

1. High efficiency. The multimodal design improves filtering accuracy (discussed in Section 3) while maintaining high efficiency. If a requested site appears in the URL cache, the filtering decision can be made quickly (in less than one micro second). Otherwise, the site's texts and images are analyzed for accurate site classification and the URL-cache is updated. In addition, *MORF*'s text/image analysis does not consume significant computational resources.
2. High accuracy. *MORF* uses the result of the text analysis to set the *bias threshold* for conducting image analysis. If the text analyzer suspects that the site is objectionable, the image analyzer is set biased toward classifying the site's images as objectionable. The image analyzer serves as a lexical disambiguator. In addition, *MORF* employs a novel *Cross-Bagging* ensemble scheme (discussed in Section 3.1) to achieve accurate image-class prediction. The employment of the multimodal analysis and the novel classifier enables *MORF* to make accurate Web-page classification.
3. Personalization. *MORF* uses the perception-based learning algorithms [1] to adaptively formulate filtering criteria to satisfy different needs of individuals.

The rest of this paper is organized as follows. In Section 2 we depict the components of *MORF* and key system events. In Section 3 we discuss *MORF*'s multimodal classification algorithm (MCA) and its concept-shift algorithm

[1] In this paper, unless otherwise stated, objectionable information refers to undesirable and offensive information that can be personally defined; e.g., pornography, hate messages, etc.

(CSA) for personalization support. In Section 4 we discuss the distributed site database update mechanism among multiple *MORF* systems. Section 5 presents experimental results and some statistics collected from the deployed sites in the United States and Asia. Finally, we present our concluding remarks in Section 6.

2 System Components and Events

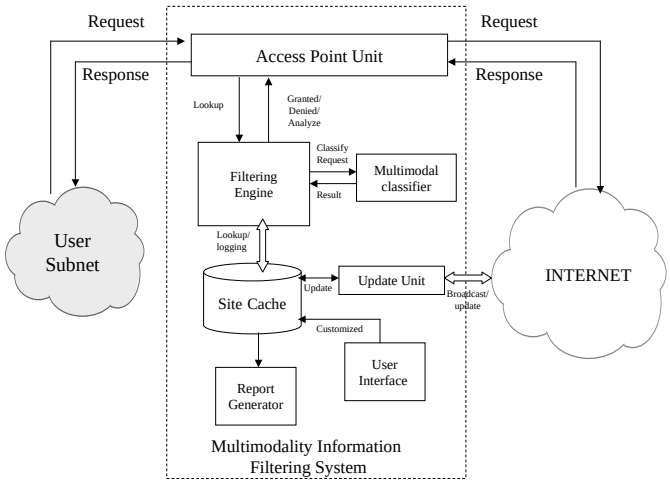


Fig. 1. System Architecture of our Multimodality Personalized Information Filter

Figure 1 presents *MORF*'s functional units including *access point unit*, *filtering engine*, *site cache*, *multimodal classifier*, *update unit*, *user interface*, and *report generator*. The *access point unit* receives in-coming requests to *MORF*. A received request is forwarded to the *filtering engine*, which decides whether the request should be granted or denied by first looking up in the *site cache* (i.e., the URL cache). If the requested site does not appear in the URL cache, the *filtering engine* 1) queries the *multimodal classifier* to classify the site, 2) updates the site cache, and 3) returns the permit/deny decision. The *update unit* of *MORF* multicasts and synchronizes new URL entries to and from the other *MORF* systems. The *user interface* allows users to provide feedback for personalizing filtering criteria. Finally, the *report generator* generates traffic statistics.

The filtering decision must be made both swiftly and accurately. *MORF* employs three policies to strike a good balance between high filtering accuracy and efficiency.

1. *Presumed pass.* If a request encounters a cache miss, *MORF* first permits the unknown site to be accessed by the user, and at the same time, it classifies the site asynchronously. If the presumed-pass request happens to be objectionable, any subsequent access to that site will be blocked and a log entry made that the objectionable site has been accessed.
2. *Distributed site-cache update.* Each *MORF* periodically multicasts its new entries in the block and bypass lists to its peers. Once a new site is seen and classified by one *MORF*, the new entry is propagated to the other *MORF*s.
3. *Client vs. server deployment.* When filtering accuracy cannot be compromised, *MORF* can be deployed at the client site so that any unseen site is classified synchronously. This synchronous classification does not affect the access performance of the other clients.

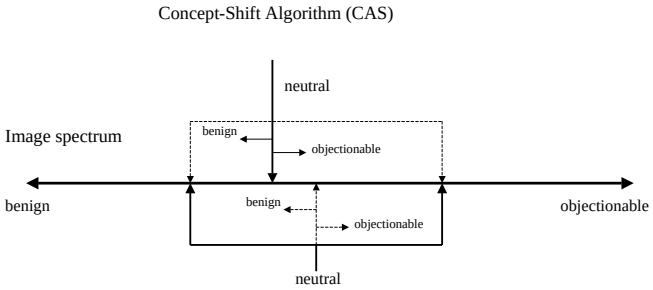


Fig. 2. MCA and CSA.

The efficiency and effectiveness of *MORF* relies on its multimodal classification and concept-shift algorithms, which we discuss next.

3 Classification Algorithms

To support accurate and personalized site classification for *MORF*, we propose two algorithms, the *multimodal classification algorithm* (MCA) and the *concept-shift algorithm* (CSA).

1. *Multimodal classification algorithm (MCA).* MCA first analyzes the requested site's text information and then its image content to predict the site's class, either *benign* or *objectionable*. The text classifier returns a score indicating the probability that the requested site is objectionable. This score is used by the image classifier to set the *bias threshold*. The image classifier classifies an image into one of the three categories: *benign*, *neutral*, and *objectionable*. The more the *bias threshold* leans toward the objectionable side, the more the neutral-objectionable boundary is shifted away from the objectionable side, and therefore, the more likely a neutral image will be classified as an objectionable one. On the other hand, the more the *bias threshold* leans toward the benign side, the more likely a neutral image will be classified as benign. Finally, MCA aggregates image scores to predict the class of a given site. We present MCA in details in Section 3.1.
2. *Concept-shift algorithm (CSA).* Based on individual subscriber feedback, CSA can further classify the neutral category predicted by MCA into either the *benign* or *objectionable* class. In other words, CSA places a *personalized view* on the neutral category. (Note that without CSA, MCA splits the neutral category into benign and objectionable categories by setting the boundary at the middle of the neutral category.) For instance, a high school may permit accessing some sites which an

elementary school denies, and CSA can assist in personalizing the filtering criteria. We present the details of CSA in Section 3.2.

Figure 2 explains how MCA and CSA work. The bottom of the figure shows that the bias threshold of MCA affects the boundary between the benign class (on the left-hand side) and the objectionable class (on the right-hand side) by demarcating a fence in the neutral region. The top of the figure shows that CSA draws a personalized boundary in the neutral region based on individuals' preferences. The MCA provides a default rating for a site, and the CSA sets personalized ratings for individuals. For instance, if a site's texts indicate that the site does not contain objectionable words, the text classifier sets the bias threshold toward benign (the fence in the neutral zone at the bottom thus lies closer to the objectionable side). The site is therefore less likely to be classified as objectionable unless its images provide stronger than usual evidence. The personalized threshold can override the system's default classification. For example, a conservative institute that does not want to take any chances of viewing objectionable content can set a very strict threshold close to or even within the benign boundary.

3.1 Multimodal Classification Algorithm

We employ a novel classification scheme, *Cross-Bagging* ensemble, for MCA. The design goals of the Cross-Bagging ensemble are threefold: fast training time, fast classification time, and high classification accuracy. The MCA consists of three parts: the Bayes Point Machines (BPM) base classifier, the ensemble scheme, and the text, image and site classifiers. However, due to space limitation, we only discuss the BPM base classifier and the ensemble scheme in this paper. Detailed discussions of the text, image and site classifiers can be found in the extended version of this paper [2].

The BPM Base Classifier Recently, Gaussian processes have been adapted to the problem of classification. The Bayes Point Machines (BPM) scheme, based on the Bayesian learning theory, considers learning as an optimal sampling and voting scheme on the Bayesian posterior. With this background, the BPM was proposed by Herbrich [3, 4] to approximate Bayesian inference for linear classifiers in a kernel space. In contrast to the SVM, the BPM constructs the hypothesis (functions to map input vector x to its corresponding output y) using the whole version space (a space of all consistent hypotheses given a training set) and finds the midpoint of the region of intersection of all hyperplanes bisecting the version space. This midpoint is known as the Bayes point. This classifier (the Bayes point) could be approximated by the center of mass [5] which Ralf [3, 4] proposed to estimate by using both a dynamic system that averages over the trajectory of a billiard ball bouncing in version space, and an approximate method that uses the perceptron algorithm trained on permutation of the training set.

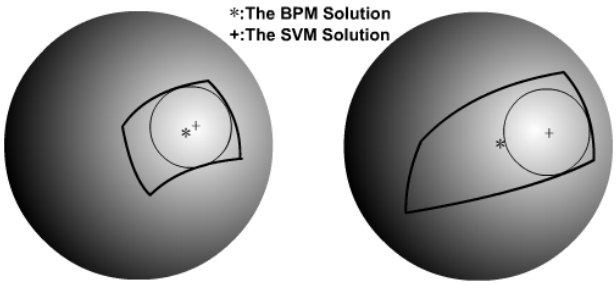


Fig. 3. Version space $V(z)$ for a 3D toy dataset. From the Bayesian perspective, one can see that the SVM approximates the Bayes point (the star symbol, trained by the BPM) by the center of the largest inscribable ball (the addition symbol) in the version space. SVM could be way off the center-of-mass.

The major difference between the BPM and the SVM, from a Bayesian perspective, the SVM can also be viewed as an approximation of the center of mass in the version space [3]. The SVM chooses the hypothesis corresponding to the largest possible hyperplane that can be the training points at which the largest inscribable ball touches the corresponding hyperplane. Compared with the BPM, which uses the whole version space to construct the hypothesis, the SVM would obtain unreasonable classifiers if this inscribable ball did not cover most volumes of a version space with an elongated or irregular shape. Figure 3 illustrates this key difference. Detailed discussion of the BPM classifiers versus SVM classifiers can be found in [6].

Cross-Bagging Ensemble One can employ the BPM base classifier to perform binary class prediction on text documents or on images. But the classification accuracy of a binary BPM classifier can certainly be improved. Bagging [7] and Boosting [8, 9] are two well-known and extensively validated methods for improving classification accuracy of a base classifier. However, we will explain shortly that a straight adoption of Bagging and Boosting can be computationally intensive, and thus not practical in an online application like *MORF*. We therefore propose Cross-Bagging, which intelligently combines Bagging and Boosting to keep the computational cost down.

Bagging employs multiple versions of a base classifier and has been shown successful in reducing classification variance. Boosting adds new classifiers to an ensemble by training the base classifier sequentially to reweighted versions of the training dataset so that the next classifier will focus more on misclassified examples (and less on the correctly classified examples). Bagging adds classifiers that

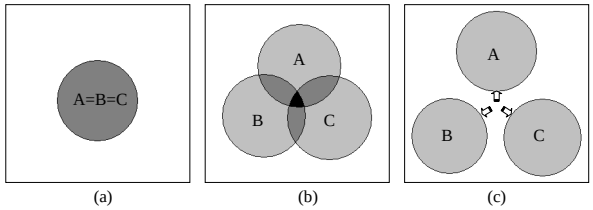
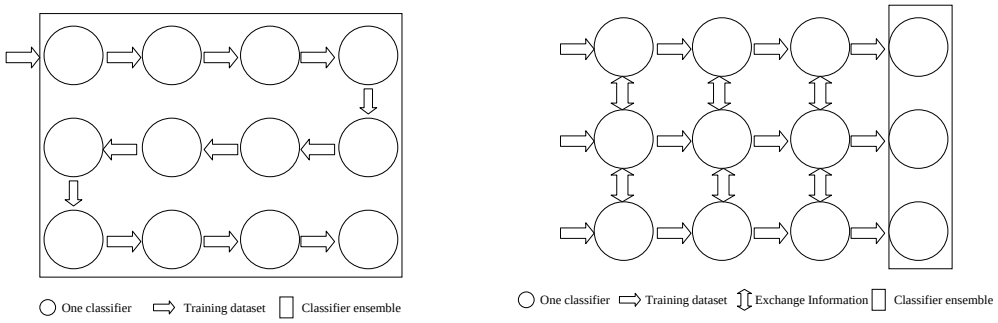


Fig. 4. Dark regions are the portions of a population that are misclassified by the ensemble. Gray regions are the portions of the population that are misclassified by individual classifiers. Case (a): A, B, C are identical. Case (b): A, B, C are independent. Case (c): A, B, C are mutually exclusive.



(a) Traditional Boosting (b) Cross-Bagging

Fig. 5. The difference between traditional Boosting and Cross-Bagging

aim to make independent errors, and Boosting adds classifiers that aim to make mutually exclusive errors. Figure 4 illustrates the difference between Bagging and Boosting. The circle in Figure 4(a) shows the classification error of one BPM base classifier. When Bagging is employed and three bags are used, Figure 4(b) shows that the error (the overlapped area) made by three independently constructed bags can be smaller than the error of an individual classifier. When Boosting is employed, Figure 4(c) shows that the errors made by different bags could be mutually exclusive, thus reducing the prediction error of the ensemble. Boosting, however, is not computationally feasible for an on-line application such as *MORF*, since it often requires training a large number of classifiers (possibly in the order of thousands), which results in a large ensemble size. A large ensemble size prolongs both training time and classification time [7, 8, 9, 10].

Our *Cross-Bagging* scheme aims to improve classification accuracy by intelligently selecting training data for each bag. The key idea of Cross-Bagging lies in the observation that we need only the majority of the ensemble members to vote correctly to make an accurate class prediction. Let G denote the number of bags, Γ the training dataset, T the testing dataset, and B_i the BPM classifier of the ith bag, where $i = 1$ to G. Let BPM_{Train} denote the BPM training procedure, and $\text{BPM}_{Classify}$ the BPM classification procedure. The scheme works as follows:

1. *Initialization.* Randomly select training samples τ_i for each bag, and train an BPM classifier B_i for each bag.
2. *Step 1.* Take a set of testing samples τ_t (from the training dataset) in the tth iteration to test the trained ensemble B, where B consists of G bags.
3. *Step 2.* The ensemble of G bags[2] classifies τ_t and yields a misclassified set τ_t^-.
4. *Step 3.* For each misclassified test instance α in τ_t^-, insert the instance into the training dataset $\tau_i$of the randomly selected $\lceil G/2 \rceil$ bags. Note that we insert the misclassified instances into only a majority of bags to ensure that the instances will not be misclassified again in the subsequent iterations.
5. *Step 4.* Train a new BPM classifier B_i on the updated training dataset τ_i for each bag.
6. *Step 5.* Test the classifier ensemble on the testing dataset T and yield a misclassified subset T^-.
7. *Step 6.* Update statistics. If the testing error has been reduced from the previous iteration, go to *Step 1*, or terminate the algorithm.

Why Cross-Bagging works and how it differs from Boosting? The traditional Boosting method reweights the training population, according to the testing results in the previous round, to train another classifier. Figure 5(a) shows how the traditional Boosting scheme trains a series of classifiers and how a class prediction involves the voting of all classifiers (in the rectangle). Cross-Bagging differs from Boosting in two ways, as depicted in Figure 5(b).

1. Mutual exclusive soft boosting. Cross-Bagging boosts only enough bags to ensure that a misclassified instance will be correctly classified in the next iteration. For instance, suppose we have three bags, A, B, and C, and that instance x_i is misclassified by bags A and B in the current round. We boost x_i in only one of bags A and B in the next training iteration to ensure that the majority will make mutually exclusive errors on x_i and thus can predict x_i correctly.
2. Small voting members. The class prediction of Cross-Bagging is made by the last set of bags. Figure 5(b) shows that only the last set of bags (in the rectangle) is involved in class prediction, and hence the classification time remains constant, independently of the number of rounds of boosting.

3.2 Concept-shift Algorithm

MORF provides personalization support via the Concept-shift Algorithm (CSA). A concept shift is detected when a site manager (e.g., a parent) disagrees with the default class prediction of *MORF*. *MORF* provides a user-interface which allows a site manager to examine boundary sites (i.e., the sites in the neutral zone depicted in Figure 2) and

[2] Figure 8 shows an implementation of the voting scheme among G bags. Our experimental results with different voting formulas do not show significant differences between them, and hence we do not discuss the details of the voting procedure.

provide relevance feedback. What CSA does is to learn a personalized classifier for the neutral zone.

CSA consists of the following steps:

Step 1. Present K random sites from the boundaries of the neutral zone to the user to mark for one of the three ratings: benign, objectionable, or neutral.

Step 2. Train three one-per-class (OPC) binary classifiers, one for benign and objectionable, one for benign and neutral, one for objectionable and neutral. Detailed discussion of this active learning algorithm can be found in [1].

Step 3. Sample sites and present K sites that cannot be "cleanly" classified by the OPCs to solicit relevance feedback.

Step 4. Go to Step 2 unless the user terminates the algorithm.

Step 5. Output the learned bias threshold from the OPCs trained by the collected labeled instances.

4 Distributed Site Database Update

MORF systems have been installed and are running at many distributed sites. Once a MORF system discovers a new objectionable site, it propagates the URL of that site to the other MORF systems. Specifically, a MORF site-database needs to be updated in the following three scenarios:

1. An objectionable site has been discovered locally.
2. A site's class is overwritten by a local administrator.
3. A new objectionable site is informed of by a remote MORF system.

The first case of update is self-update, and it is straightforward. As described in Section 2, after MORF identifies a new objectionable site, it inserts the URL onto the local block list.

The second case of update is administrative update. An administrator can overwrite the blocking decisions made by the site classifier. An administrator can also manually enter sites for blocking or for bypassing. When more than one filtering systems reside in a local network (e.g., a cooperate network domain), they usually want to enforce the same blocking criteria without having to update each filtering system individually. To synchronize administrative updates in a local domain, a MORF system broadcasts its updates using the Multicast Transport Protocol to the other MORFs in the same local domain. A MORF system constantly listens to its multicast port and updates its site database when it receives an administrative update. A local administrative update can thus be done at any MORF system in the local domain.

The third kind of update is global master-list update. MORF systems deployed at different networks are likely to discover new objectionable sites independently. Each MORF system periodically propagates it's new findings to as well as receives new URL updates from remote MORF systems. This global synchronization is accomplished by having a centralized master-list database that collects new objectionable site-URLs from all MORFs. At the same time, each individual MORF system periodically requests for an incremental update from the centralized server to keep its local list up-to-date.

As the number MORF systems increases, the centralized master-list database can become the bottleneck of the MORF's global synchronization scheme. To alleviate this problems, we employ three load-balancing strategies.

- First, instead of updating the master list at the same time, each MORF system updates at a randomly selected time. This approach distributes the update traffic in the temporal dimension.
- Second, the update list is mirrored at multiple update servers. Each MORF system randomly selects two servers to request for an incremental list, and the update is performed by the less busy site. According to [11], this randomized scheduling scheme ensures load balancing in high probability. Our prior work [12] and empirical studies both confirm its effectiveness.
- Finally, if there are multiple MORF systems in a local domain, only one MORF system is needed to get the incremental update list from the centralized server. The MORF system that holds the update list can synchronize the update with the other MORF systems in the same local domain. This hierarchical update scheme takes advantage of the broadcasting scheme used in the administrative update scenario. This approach reduces the number of update requests to the servers.

In summary, we present a distributed update scheme for synchronizing the site database in individual MORF systems. By collaborating many filtering systems that can discover objectionable sites independently, this update scheme can provides the most comprehensive and most up-to-date blocking Website list.

5 Experiments

Our empirical study was design to answer the following three questions:

1. How many bags should be used and what sampling strategy works the best for Bagging?
2. Can Cross-Bagging outperform the traditional Bagging scheme in classification accuracy and training time?
3. Does a multimodality classifier perform better than single modality classifiers in filtering objectionable Websites?

In addition, we present at the end of this section some statistics we collected from our beta sites in the U.S. and Asia.

5.1 Experimental Settings for Image Classification

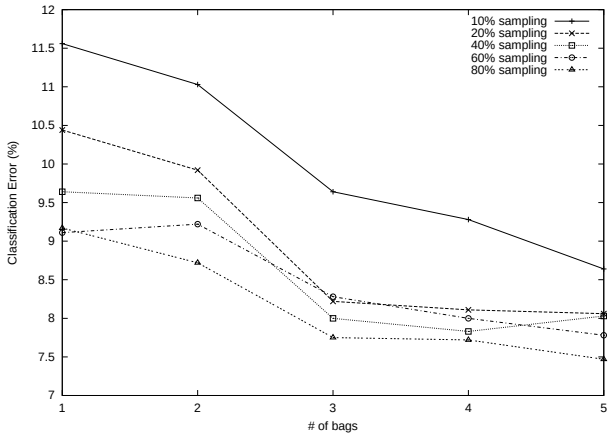
The dataset used in our experiments consisted of 20,000 pornographic images as positive examples and 20,000 benign images as negative examples. All images were obtained

by crawling the Internet and randomly selected after manually identifying their classes (objectionable/benign). Ten percent (4,000 images) of the entire image population were randomly selected as the testing set and separated from the rest. The remaining 36,000 images were used as the training population.

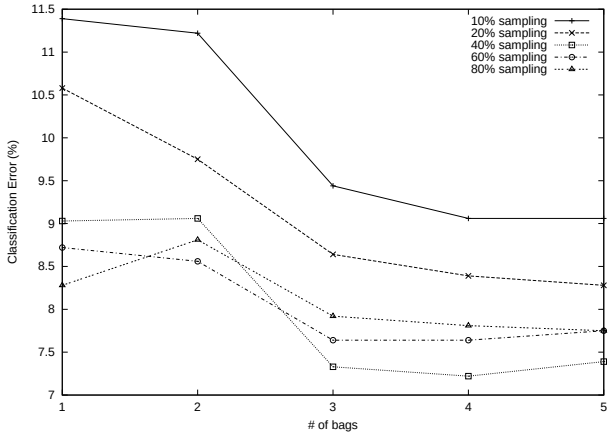
For each image, 144 image features were extracted as discussed in [13, 1]. Different BPM kernel functions were evaluated. The radial basis function kernel, which worked consistently more accurately than the polynomial basis function kernel, was chosen to work with the BPM base classifier.

5.2 Experimental Results of Bagging with Different Sampling Criteria

In this section we present experimental results of traditional Bagging with different sampling criteria: sampling with and without replacement.



(a) sampling with replacement



(b) sampling without replacement

Fig. 6. Classification Error with Bagging

The sampling-with-replacement policy is that after a population member has been drawn as a sample, this subject is put back into the population pool. All N members of the population have equal probability of being selected at each drawn, even if they have already been drawn. In contrast, the sampling-without-replacement policy is that once a population member has been drawn, this subject is removed from the population pool for all subsequent draws. Therefore, once a population member has been drawn, the probability that other members in the population get selected in subsequent draws is increased. Note that in sampling-with-replacement policy, the samples are independent. This means that what we get on the first one doesn't affect what we get on the second. In sampling-without-replacement policy, the samples are not independent. Practically, this means that what we got for the first one affects what we can get for the second one.

First we experimented to decide which sampling policy produces more accurate classification results. Figure 6(a) shows the accuracy of the Bagging algorithm on the testing set using the sampling-with-replacement policy and Figure 6(b) using the sampling-without-replacement policy. The number of bags varies from one to five and the training sample ratio varies from 10% to 80% of the training population for each bag. Each training bag is independently constructed by applying the respective sampling policy on the training population. The results in Figure 6(a) and (b) show that the accuracy of the sampling-without-replacement policy is greater than that of the sampling-with-replacement policy in almost all cases. We conclude that sampling-without-replacement policy produce more accurate classification results in our application.

Next we try to find the best combination of training sample ratio and number of bags. Figure 6(b) shows that increasing training data does not necessarily lead to higher class-prediction accuracy. Figure 7 shows the same results as in Figure 6(b) but with respect to the training sample ratio. In Figure 7, the lowest classification error (7.4%) is produced with a training sample ratio of 40%. Also, the improvement in accuracy beyond 3 bags seems to be marginal. The results also show that increasing the training sample ratio does not further reduce classification errors in these cases. This phenomenon confirms the recent finding in [14], which shows that half-sampling without replacement policy produces the least amount of classification error and that it is virtually equivalent to traditional bootstrapping with 100% training sample ratio with replacement policy.

From the above experiments, we found the answer to the first question raised in the beginning of this section. In our application, the Bagging algorithm works very well with three bags, a training sample ratio of 40%, and with training sets generated by the sampling-without-replacement policy.

5.3 Experimental Results of Cross-Bagging

Figure 8 shows two sets of experimental results of the proposed Cross-Bagging algorithm. One experiment started the training with a training sample ratio of 10% and the other experiment with a sample ratio of 40%. The testing set was 10% of the total population and was separated

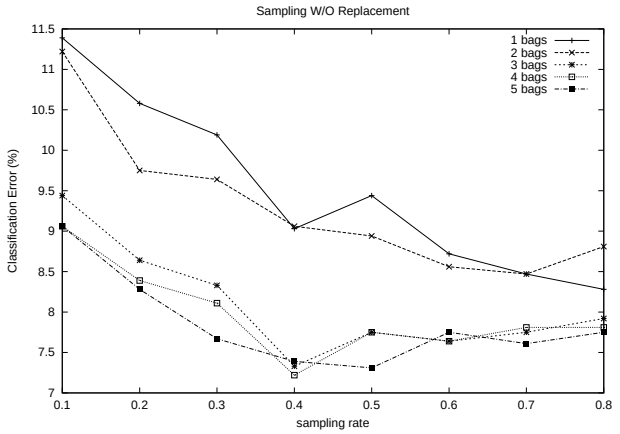


Fig. 7. Classification Error of Bagging with Sampling-Without-Replacement Policy

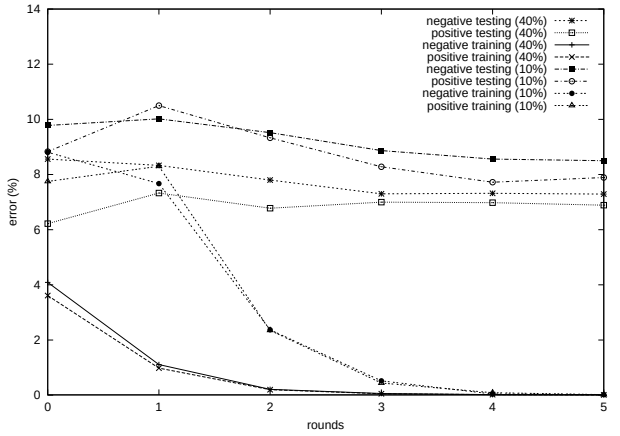


Fig. 8. Training and Classification Error in Cross-Bagging Algorithm

from the training sets as discussed in Section 5.1. Three bags were used in both experiments. We applied the Cross-Bagging algorithm for five rounds on both experiments. First, Figure 8 shows that the false-positive error (negative examples classified as positive) and the false-negative error (positive examples classified as negative) are comparable in all cases. Second, in Figure 8, the training errors in both experiments decreased logarithmically with each round for the first 3 to 4 rounds. The results show that after a small number of iterations, the training errors become negligible. The testing errors in both cases also decrease accordingly, but at slower rates. The results suggest that the proposed Cross-Bagging algorithm requires only a few training iterations (in our application, three iterations) to achieve accurate class-prediction.

The next experiment compared the training costs of the proposed Cross-Bagging algorithm to those of the traditional Bagging algorithm. Figure 9 shows the training time of the traditional Bagging algorithm and the proposed Cross-Bagging algorithm. A training sample ratio of 40% was used for the traditional Bagging algorithm and a training sample ratio of 10% was used as the starting training set size of the Cross-Bagging algorithm. We ran the Cross-Bagging algorithm for three rounds and compared the total training time with the traditional Bagging algorithm, which runs in a single round. The training time was wall clock time in seconds. Figure 9 shows that the total training time of the proposed Cross-Bagging algorithm is almost 50% less than the training time of the Bagging algorithm.

From the above experiments, we answered the second question raised in the beginning of this section. The proposed Cross-Bagging algorithm requires much less training overhead to achieve comparable accuracy than the traditional Bagging algorithm.

5.4 The Advantage of Multimodality Design

The Web-page filtering accuracy can be increased by using the image classifier as a lexical disambiguator, as discussed in Section 3. The employment of the multimodal analysis and the novel classifier enables the proposed information filtering system to make accurate Web-page classification. The next experiment compared the Web-page filtering accuracy of different filtering techniques discussed in Section 1. This experiment used a 30 days web access logs from MORF testing sites in the United States and Asia. The logs contained a history of 103.2M Web access requests to 25,371 different Web-sites. Out of 25,371 Web-sites, 1,266 sites were considered to be pornographic sites.

We compared the filtering accuracy of URL-based, keyword-based, image-based, and the proposed multimodality techniques on the access requests to the objectionable sites. The result in Figure 10 shows that using the URL blocking list alone (from an unnamed commercial filtering software) produced filtering error greater than 50%. The reason for the high filtering error is that the URL-based technique relied on a block list which is often out of date. In contrast, the keyword-base technique produces 33.4% filtering error in this experiment. In this case the high filtering error resulted because many of the objectionable sites contain no keywords. Another reason is that many of those objectionable Web-pages in this experiment were written in languages other than English. The low accuracy of the keyword-based technique strongly suggests the use of information other than text content for filtering purposes. The images-based filtering technique use only images from a site to determine if the site contains objectionable in-

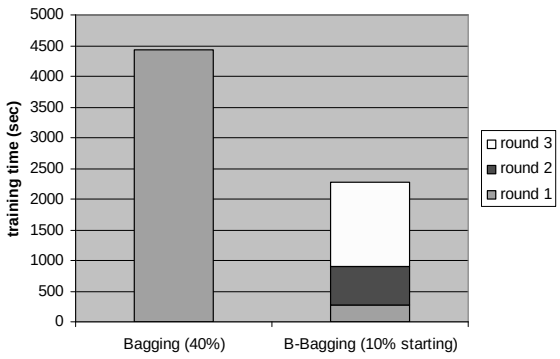


Fig. 9. Training Time Comparison

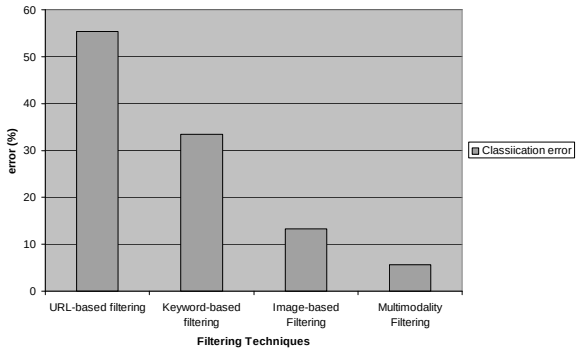


Fig. 10. Filtering Accuracy Comparison

formation. The images-based filtering technique produces 13.3% filtering error, which is much lower than the text-based technique, in this experiment. This result suggests that image content is more reliable than text content in the filtering applications because images are less ambiguous to interpret. By employing the multimodality design, our proposed filtering technique can further reduce the classification error. In this experiment, our proposed multimodality filtering system produced the least filtering error, 4.6%, compares to other filtering techniques. This result answers the third question raised in the beginning of this section: the proposed multimodal classification technique does outperform other single modality classifiers in filtering pornographic web-sites.

6 Conclusion

In this paper, we propose a personalizable information filtering system with multimodality design. We depict the components of the filtering system and show how the filtering components interact with each other. The proposed system employs three filtering models— URL, text, and image content—to accurately determine whether a site is objectionable and thus blocks users access to the objectionable sites. We empirically showed that sharing information among different models improves the overall accuracy of classifying web sites with objectionable contents. The proposed multimodality information filtering system is flexible for personalization and is ideal for ISP's, home, schools, libraries, and corporate users to ensure adequate level of filtering criteria in real time.

Acknowledgements

The authors would like to thank Huaxin You for helpful and inspiring discussions.

References

[1] Simon Tong and Edward Chang. Support vector machine active learning for image retrieval. *Proceedings of ACM International Conference on Multimedia*, pages 107–118, October 2001.

[2] Yi-Leh Wu, Edward Y. Chang, Kwang-Ting Cheng, Cheng-Wei Chang, Chen-Cha Hsu, Wei-Cheng Lai, and Ching-Tung Wu. MORF: A distributed multimodal information filtering system (extended version). *Technical Report http://www.morphpsoft.com/whitepapers/morf-extended.pdf*, June 2002.

[3] Ralf Herbrich, Thore Graepel, and Colin Campbell. Bayes point machines. *Journal of Machine Learning Research*, 1:245–279, 2001.

[4] Ralf Herbrich, Thore Graepel, and Colin Campell. Bayes point machines: Estimating the bayes point in kernel space. *Proceedings of IJCAI Workshop Support Vector Machines*, pages 23–27, 1999.

[5] T. Watkin. Optimal learning with a neural network. *Europhysics Letters*, 21:871–877, 1993.

[6] Gang Wu, Edward Chang, and Chung-Sheng Li. Bpms vs svms for image classification. *Proceedings of IEEE Multimedia and Expo*, August 2002.

[7] L. Breiman. Arcing classifiers. *The Annals of Statistics*, 26(3):801–849, 1998.

[8] J. Friedman, T. Hastie, and R. Tibshirani. Additive logistic regression: a statistical view of boosting. *Dept. of Statistics, Stanford, University Technical Report*, 1998.

[9] Robert E. Schapire, Yoav Freund, Peter Bartlett, and Wee Sun Lee. Boosting the margin: a new explanation for the effectiveness of voting methods. In *Proc. 14th International Conference on Machine Learning*, pages 322–330. Morgan Kaufmann, 1997.

[10] T. G. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artificial Intelligence Research*, 2:263–286, 1995.

[11] Y Azar, A Broader, A Karlin, and E Upfal. Balanced allocations. *ACM Symposium on Theory of Computing*, pages 593–602, December 1994.

[12] Edward Chang, Hector Garcia-Molina, and Chen Li. 2d bubbleup - managing parallel disks for media servers. *Proceedings of the 5th Foundations on Data Organization*, November 1998.

[13] Beitao Li, Edward Chang, and Chung-Sheng Li. Learning image query concepts via intelligent sampling. *Proceedings of IEEE Multimedia and Expo*, August 2001.

[14] Jerome H. Friedman and Peter Hall. On bagging and nonlinear estimation. *http://www-stat.stanford.edu/~jhf/ftp/bag.ps*, 1999.